

ДЗ 6 · ПРОЕКТНАЯ РАБОТА ПО SYSTEM DESIGN

Мессенджер на 1 млрд пользователей

Архитектура, расчёты нагрузки и стоимости, ключевые решения

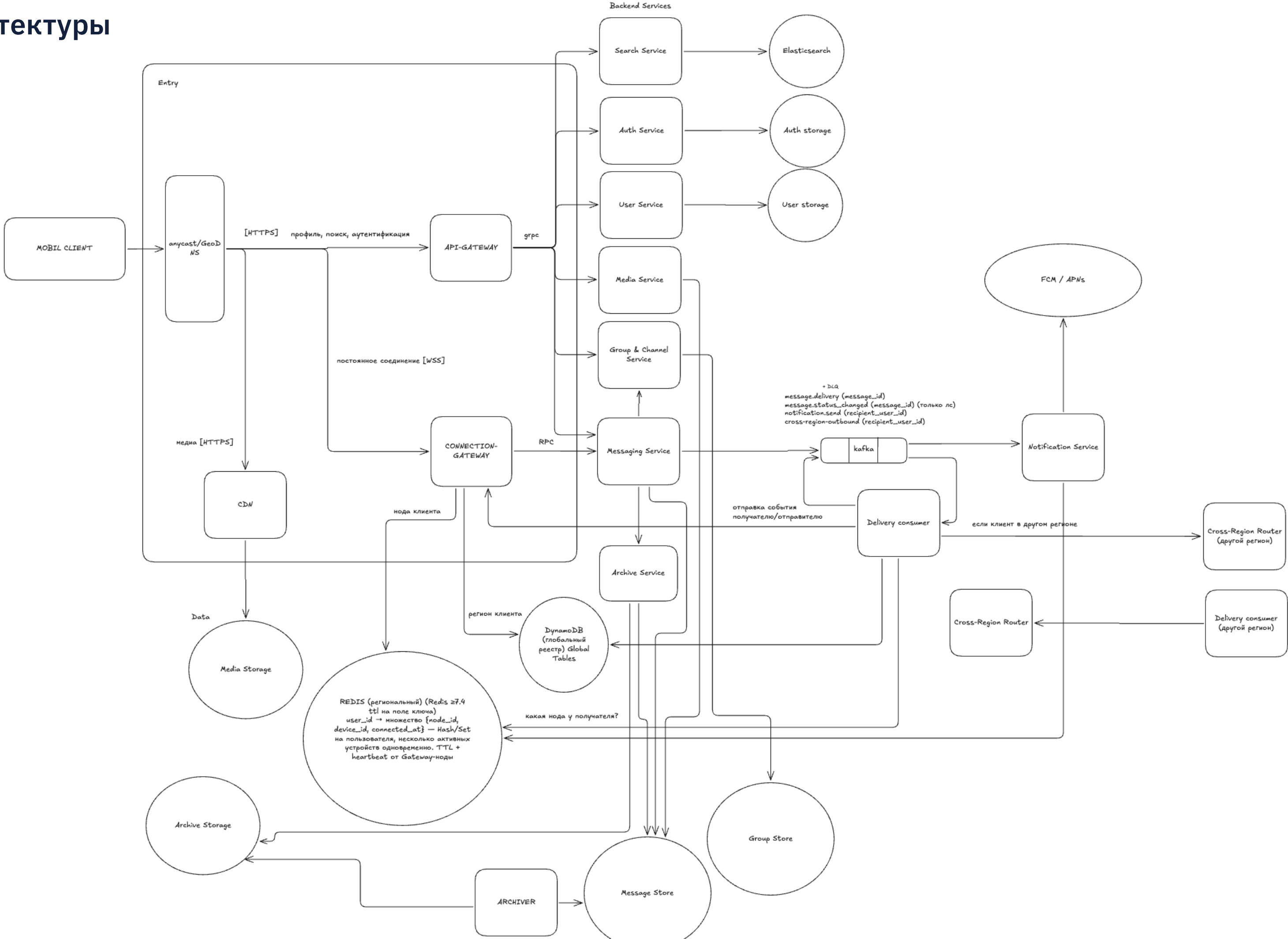
ИСХОДНЫЕ ДАННЫЕ

Требования, от которых считается всё

1 млрд пользователей в месяц (MAU)	500 млн пользователей в день (DAU)	45 сообщений на DAU в день; 20% — медиа до 2 ГБ	200 000 участников в группе; каналы до 100 млн
99,95% SLA; отказ региона — не дольше 90 сек	≤ 200 мс p95 доставки в регионе, ≤ 1 с между регионами	3 региона EU, US, APAC — пользователи по всему миру	Бессрочно хранение переписки; поиск — на 1 год

Пик: 781 тыс. сообщений/сек · 39,6 млн соединений · 15,3 Тбит/с отдачи медиа

Схема архитектуры



Региональная модель: три равных региона

×9 копий сообщения · запас ×1,5 на отказ региона · 30–90 сек на переподключение

Active-active: все три региона равны

КОНТЕКСТ

Запись — до 200 мс в любом регионе, а primary + DR даёт +100–200 мс двум регионам из трёх.

РЕШЕНИЕ

Полная копия RF3 в каждом регионе, запись и чтение локально (LOCAL_QUORUM), promote не нужен.

КОМПРОМИСС

×9 копий вместо ×5 — +\$6,7 млн/мес (4,9% счёта); секунды лага между регионами.

Запас ×1,5 заложен заранее

КОНТЕКСТ

При отказе региона двое выживших получают по половине его трафика — ×1,5 на каждый.

РЕШЕНИЕ

CG, Redis, Messaging и Kafka держат ×1,5; запасной регион клиент выбирает по hash(user_id).

КОМПРОМИСС

~470 инстансов простаивают штатно — ≈ \$137 тыс./мес, 0,1% счёта.

Переподключение в две фазы

КОНТЕКСТ

13 млн клиентов за 30 сек перегрузят DynamoDB, Auth и чтение истории в 5–10 раз.

РЕШЕНИЕ

Подключение со случайной паузой 0–60 сек, синхронизация — в фоне за ~5 мин.

КОМПРОМИСС

Пользователи упавшего региона 30–90 сек без связи, полная синхронизация — до 5 мин.

СЦЕНАРИЙ 1

Личное сообщение в своём регионе

p95 ≤ 200 мс · 507 тыс. личных сообщений/сек в пик

- 1 **Клиент А → Connection Gateway** — send_message: peer_id собеседника, client_message_id и last_seen_hlc
- 2 **Messaging → Redis Sequencer** — номер в потоке (чат, регион) и текущий бакет
- 3 **Messaging** — гибридная метка больше всего, что видел автор (не дальше now + 5 сек): ответ всегда после вопроса
- 4 **Messaging → Cassandra** — запись в партицию (чат, регион, бакет); первое сообщение добавляет чат в список чатов обоих
- 5 **Messaging → Kafka** — событие с recipient_id в message.delivery; клиенту А — «отправлено»
- 6 **Delivery consumer → Redis** — HGETALL: все устройства получателя и другие устройства отправителя
- 7 **Connection Gateway → клиент В** — событие на каждое устройство; «доставлено» и «прочитано» уходят статусом отправителю

Если что-то пошло не так

Kafka недоступна — «отправлено» не приходит, клиент повторяет с тем же client_message_id

Получатель офлайн — кандидат на push: не чаще 1 push на чат в 60 сек

Получатель в другом регионе — межрегиональная доставка, сценарий 2

СЦЕНАРИЙ 2

Личное сообщение в другой регион

$p95 \leq 1 \text{ с} \cdot 546 \text{ тыс. межрегиональных событий/сек в пик}$

- 1 **Запись** — как в сценарии 1, локально в регионе отправителя (EU)
- 2 **Delivery consumer (EU) → Redis EU** — промах: в EU у получателя нет открытых устройств
- 3 **Delivery consumer → DynamoDB** — регионы с открытыми устройствами получателя: {US}
- 4 **Delivery consumer → Kafka** — событие в cross-region-outbound для US
- 5 **Cross-Region Router EU → US** — прямой gRPC; offset сдвигается только после успеха
- 6 **Router US → Redis US → Connection Gateway US** — доставка на устройства клиента В
- 7 **Cassandra** — сообщение асинхронно реплицируется в US и APAC за секунды

Если что-то пошло не так

Сбой gRPC — offset не сдвинут, Kafka отдаёт событие повторно

Регион получателя недоступен — получатель считается офлайн, push уходит из EU

Получатель офлайн везде — кандидат на push через окно агрегации

СЦЕНАРИЙ 3

Сообщение в группу до 1 000 участников

273 тыс. сообщений/сек · ~30 получателей · 8,2 млн поисков/сек

- 1 **Автор** → **Connection Gateway** → **Messaging** — send_message в группу
- 2 **Messaging** → **Group & Channel** — CheckAccess: автор состоит в группе
- 3 **Messaging** → **Sequencer** → **Cassandra** — номер, метка HLC и одна запись в поток группы
- 4 **Messaging** → **Kafka** — одно событие в message.delivery
- 5 **Delivery consumer** → **Group & Channel** — участники из кэша cache:group; для каждого — онлайн ли он в Redis
- 6 **Онлайн** — через Connection Gateway; в другом регионе — через Cross-Region Router
- 7 **Офлайн** — +1 к счётчику в push:{user_id}; push не уходил 30 мин — дайджест через Notification Service

Почему так

Рассылка при записи дешёвая — не больше 1 000 поисков на сообщение

Исключение сразу — кэш списка участников сбрасывается при любом изменении состава

Push по группам — дайджест не чаще раза в 30 мин; упоминания — как личные

СЦЕНАРИЙ 4

Большая группа или канал

до 200 000 участников и 100 млн подписчиков · стоимость — по нодам CG, а не по людям

- 1 **Участник открывает чат** → **Connection Gateway** — subscribe_chat после проверки доступа
- 2 **Connection Gateway** → **Redis** — нода CG добавляется в подписчики чата subs: {chat_id}
- 3 **Клиент** → **GET /messages** — хвост ленты из кэша tail:{chat_id}, TTL 5 сек
- 4 **Автор** → **Messaging** — CheckAccess: роль автора (в канале пишут admin/owner); одна запись и одно событие в Kafka
- 5 **Delivery consumer** → **subs:{chat_id}** — по одному вызову на каждую ноду CG с подписчиками
- 6 **Cross-Region Router** — одно событие в каждый другой регион, там то же по своим подписчикам
- 7 **Нода CG** → **свои подписчики** — для очень активных чатов пачками раз в 100 мс

Почему так

Не открывшие чат — видят бейдж непрочитанного при открытии приложения

Push — только упоминания и ответы; в каналах push нет

Группа на 200 000 — 333 записи/сек вместо 66,7 млн поисков/сек при рассылке каждому

СЦЕНАРИЙ 5

Открытие приложения и синхронизация

60 тыс. подключений/сек · ~16 чтений Cassandra на синхронизацию

- 1 **Клиент** → **Connection Gateway** — WebSocket; JWT проверяется локально, без вызова Auth
- 2 **Connection Gateway** → **Redis** — устройство в реестре соединений user:{id}
- 3 **Первое устройство в регионе** → **DynamoDB** — регион добавляется в online_regions
- 4 **Клиент** → **POST /sync** — курсоры чатов; проверка доступа одним пакетом
- 5 **Messaging** → **Cassandra** — список чатов из READ_CURSORS: чаты с изменениями и новые для устройства
- 6 **Открыт личный чат** — статус собеседника приходит в ответе на heartbeat раз в 30 сек
- 7 **Фото и видео** — по подписанной ссылке прямо из CDN, мимо наших сервисов

Особые случаи

Новое устройство — сначала GET /chats (список чатов), история — при открытии чата

Пропуск в номерах — клиент видит дыру и дозапрашивает, потерь нет

Закрыто последнее устройство — регион убирается, записывается last_seen

СЦЕНАРИЙ 6

Листание истории старше года

~3,5 тыс. страниц/сек из архива • p95 ≤ 500 мс

- 1 **Клиент** → **GET /messages?before** — вектор курсора по регионам, 50 событий назад
- 2 **Group & Channel** — проверка доступа; личный чат — пересчёт chat_id по peer_id
- 3 **Messaging** → **CHAT_BUCKETS** — для каждого потока: бакет горячий или в архиве
- 4 **Горячий бакет** — Cassandra, таблица своего месяца
- 5 **Архивный** → **Archive Service** — индекс блока из кэша, чтение страницы ~64 КБ по диапазону байт
- 6 **Archive Service** — распаковка zstd и удаление сообщений из ARCHIVE_TOMBSTONES
- 7 **Messaging** — слияние потоков по (hlc, регион, seq), подписанные ссылки на медиа

Почему так

Для клиента путь один — горячая и архивная история отличаются только задержкой

Сегменты ~256 МБ — миллиарды тихих чатов не превращаются в миллиарды объектов

Архивирует Archiver — раз в месяц, со снапшота, не нагружая живой кластер

Messaging Service

781 тыс. сообщений + 1,3 млн статусов/сек · 651 инстанс · p95 ≤ 200 мс

Порядок: номер на (чат, регион) + HLC

КОНТЕКСТ

Сообщение из другого региона видно с задержкой — курсор по времени его пропустил бы.

РЕШЕНИЕ

Свой счётчик у каждого региона, курсор клиента — вектор; порядок показа — гибридные часы.

КОМПРОМИСС

Лента — слияние до трёх потоков; опоздавшее сообщение встает выше уже показанных.

Публикация в Kafka — best-effort

КОНТЕКСТ

Запись в Cassandra и Kafka не атомарна; outbox и CDC дороги на пути в 820 тыс./сек.

РЕШЕНИЕ

«Отправлено» — только после публикации; при сбое клиент повторяет отправку с тем же id.

КОМПРОМИСС

Редкий дубль (скрыт на клиенте) или доставка только при следующей синхронизации.

Мультидевайс

КОНТЕКСТ

Сообщения, прочтения и статус «в сети» должны совпадать на всех устройствах.

РЕШЕНИЕ

Эхо своих сообщений и read_sync на все устройства; каждое синхронизируется по своему курсору.

КОМПРОМИСС

+1,6 млн поисков/сек в Redis — +12 нод, ≈ \$6 тыс./мес.

Message Store и архив истории

14 520 нод по 4 ТБ · 58 ПБ данных · запас по записи ×9,1

Партиция (чат, регион, бакет)

КОНТЕКСТ

Партиция по chat_id растёт вечно: активная группа даёт ~19 ГБ в сутки при норме до ~100 МБ.

РЕШЕНИЕ

Бакет закрывается через 100 000 событий или на границе месяца — партиция ≤ 67 МБ.

КОМПРОМИСС

Номер бакета ищется в CHAT_BUCKETS; страница на границе читает две партиции.

4 ТБ данных на ноду

КОНТЕКСТ

Кластер считается по объёму; на нодах по 20 ТБ compaction, repair и замена ноды длятся сутками.

РЕШЕНИЕ

4 ТБ данных на диске 6 ТБ (~67%), как у Discord; 16 vCPU и 64 ГБ на ноду.

КОМПРОМИСС

Много нод — Cassandra 11% счёта. Рычаг на будущее — ScyllaDB, вдвое меньше нод.

Архив истории старше года

КОНТЕКСТ

Всё горячим с 9 копиями — +50 ПБ в год; удаление по TTL создаёт массу tombstones.

РЕШЕНИЕ

Месяц старше года — в сжатые сегменты Object Storage, таблица удаляется целиком.

КОМПРОМИСС

Старая история — p95 ≤ 500 мс вместо 200; в Cassandra до 13,2 мес данных (+10%).

Connection Gateway и Cross-Region Router

39,6 млн соединений • 297 инстансов CG • 546 тыс. событий/сек между регионами

Live-роутинг между регионами

КОНТЕКСТ

Получатель в другом регионе — $p95 \leq 1$ с;
у FCM нет SLA, а квота в 31 раз меньше
потока.

РЕШЕНИЕ

Прямой gRPC между регионами с
повтором через Kafka; регионы
получателя — из DynamoDB.

КОМПРОМИСС

Ещё два компонента: роутер с circuit
breaker и глобальный реестр в DynamoDB.

Реестр соединений: хэш на пользователя

КОНТЕКСТ

У пользователя несколько устройств, и
доставка должна найти все одним
запросом.

РЕШЕНИЕ

Redis-хэш `user:{id}`: поле на устройство со
своим TTL, выборка — один HGETALL.

КОМПРОМИСС

Нужен Redis ≥ 7.4 — ограничение при
выборе managed-провайдера.

Отказ региона без накопления очереди

КОНТЕКСТ

Пока регион недоступен, события для его
получателей копились бы в Kafka без
пользы.

РЕШЕНИЕ

Связь разорвана — получатель офлайн,
push из своего региона, offset сдвигается.

КОМПРОМИСС

Вернувшиеся клиенты забирают
пропущенное синхронизацией, а не
живой доставкой.

Group & Channel Service

порог большой группы — 1 000 · 374 тыс. проверок доступа/сек · 72 инстанса

Порог 1 000: рассылка или чтение

КОНТЕКСТ

Группа на 200 000 при рассылке каждому — 66,7 млн поисков/сек, в 7,7 раза больше всей доставки.

РЕШЕНИЕ

До 1 000 — рассылка при записи; выше — один поток, чтение при открытии и live-подписка.

КОМПРОМИСС

Два пути доставки; в больших группах push только на упоминания и ответы.

Проверка доступа через кэш членства

КОНТЕКСТ

Каждое чтение закрытой группы требует проверки членства — 374 тыс./сек, больше отправок.

РЕШЕНИЕ

Полный список членств пользователя в Redis: и «да», и «нет» — из кэша; запись живёт ≤ 24 ч.

КОМПРОМИСС

≈ \$12 тыс./мес; в чужом регионе исключение из группы действует через ~1 сек.

Хвостовой кэш каналов

КОНТЕКСТ

Канал на 100 млн лежит в одной партиции: после поста её читают все подписчики сразу.

РЕШЕНИЕ

Redis-кэш последних 100 событий потока, TTL 5 сек, одно чтение Cassandra на промах.

КОМПРОМИСС

Кто не держит канал открытым, видит новый пост с задержкой до 5 сек.

Notification Service

7,08 млн кандидатов/сек → 372 тыс. push/сек · 84% квоты FCM

Агрегация push по пользователю

КОНТЕКСТ

На пике онлайн ~8% DAU: push на каждое сообщение — 7 млн/сек, в 24 раза выше потолка FCM.

РЕШЕНИЕ

Личные — не чаще 1 push на чат в 60 сек, группы — дайджест раз в 30 мин на пользователя.

КОМПРОМИСС

Групповые уведомления реже; квоту FCM нужно согласовать — в 25 раз выше дефолта.

Адаптивное окно дайджеста

КОНТЕКСТ

Квоты FCM может не хватить при росте аудитории или её снижении со стороны Google.

РЕШЕНИЕ

Отказов FCM (429) больше 1% — окно растёт с 30 до 60 мин, загрузка падает до 55%.

КОМПРОМИСС

При нехватке квоты страдает частота уведомлений, а не доставка сообщений.

Push при отказе региона и после сбоя

КОНТЕКСТ

Push собирается в домашнем регионе получателя; после сбоя копятся устаревшие кандидаты.

РЕШЕНИЕ

Домашний регион недоступен — push шлёт регион отправителя; старше 30 мин — отбрасывается.

КОМПРОМИСС

Во время отказа региона возможен дубль push — это допустимо.

User Service и Auth Service

≤ 122 тыс. смен статуса/сек • 300 млрд строк контактов • 0 вызовов Auth на запрос

Статус «в сети»: подписка на видимых

КОНТЕКСТ

Рассылка ~150 контактам — десятки млн событий/сек, опрос видимых чатов — ~13 млн чтений/сек.

РЕШЕНИЕ

Клиент подписывается на собеседников на экране; смену статуса регионы берут из DynamoDB Streams.

КОМПРОМИСС

+36 нод Redis, ≈ \$21 тыс./мес; «не в сети» — через 20 сек.

«В сети» по последнему устройству

КОНТЕКСТ

Заккрытие одного из нескольких устройств не должно делать пользователя «не в сети».

РЕШЕНИЕ

В DynamoDB — регионы с открытыми устройствами; множество пусто — «не в сети».

КОМПРОМИСС

Запись в DynamoDB ~0,5 КБ вместе с push-токенами — ~3 ТБ хранения.

JWT проверяется на гейтвеях

КОНТЕКСТ

Проверка токена через Auth на каждый запрос сделала бы его самым нагруженным сервисом.

РЕШЕНИЕ

Асимметричная подпись: гейтвей проверяет токен публичным ключом, Auth — только вход и refresh.

КОМПРОМИСС

Мгновенного отзыва нет: при revoke рвётся соединение устройства во всех регионах, токен живёт 15–30 мин.

Media Service, CDN и Object Storage

15,3 Тбит/с отдачи · 7,5 ЭБ медиа к 3-му году · 1,75 млн скачиваний/сек

Подписанные CDN-ссылки

КОНТЕКСТ

Скачивание через редирект Media Service — 1,75 млн запросов/сек и лишний RTT на каждое фото.

РЕШЕНИЕ

Ссылка с HMAC-подписью на 24 ч лежит прямо в событии; CDN сам проверяет подпись.

КОМПРОМИСС

Исключённый участник может скачать уже выданную ссылку до конца её срока.

Собственное объектное хранилище

КОНТЕКСТ

7,5 ЭБ к третьему году: на таком объёме S3 не сходится экономически (прецедент Dropbox).

РЕШЕНИЕ

Своё хранилище с erasure coding ×1,4; сама система вне scope, цена — по тарифам S3.

КОМПРОМИСС

Нужна своя команда; в расчёте — тарифы S3 для наглядности, реальная цена ниже.

DR медиа: суточный бэкап

КОНТЕКСТ

Полная копия origin во втором регионе удвоила бы Storage — крупнейшую статью после сети.

РЕШЕНИЕ

Раз в сутки — инкрементальный бэкап в Deep Archive другого региона.

КОМПРОМИСС

RPO 24 ч и долгое восстановление; экономия ≈ \$22,6 млн/мес.

Search Service и Elasticsearch

656 тыс. документов/сек · индекс 4,2 ПБ · 393 ноды в 3 кластерах

Кластер ES в каждом регионе

КОНТЕКСТ

Один кластер — ~390 нод при разумном пределе ~250 и индексация через границу регионов.

РЕШЕНИЕ

Кластер на регион (~131 нода) индексирует свои сообщения; поиск опрашивает все три.

КОМПРОМИСС

+150–250 мс к ответу поиска; допустимо при 347 запросах/сек.

Hot / warm / cold по свежести

КОНТЕКСТ

Свежее ищут на порядки чаще, а hot-нода держит лишь ~2 ТБ и ограничена скоростью индексации.

РЕШЕНИЕ

14 дней — hot, до 90 дней — warm, до года — cold на дешёвых дисках.

КОМПРОМИСС

Нода больше (~384 против ~212 при плоском расчёте), но каждый ТБ дешевле.

Индекс — производные данные

КОНТЕКСТ

Поиск не на критическом пути доставки, а источник истины — Cassandra.

РЕШЕНИЕ

Индекс не копируется между регионами; при потере региона его часть переиндексируется.

КОМПРОМИСС

Пока идёт переиндексация, поиск по сообщениям упавшего региона неполный.

Kafka

2,75 млн событий/сек · 42 брокера · диск 49% штатно, 74% при отказе региона

Ключ партии — message_id

КОНТЕКСТ

По chat_id активная группа легла бы на одну партию и одного consumer'a.

РЕШЕНИЕ

Ключ — message_id исходного сообщения, у правки и удаления тоже: события одного сообщения идут по порядку; порядок ленты — в seq и HLC.

КОМПРОМИСС

События одного чата разбросаны по партициям — отлаживать сложнее.

Retention по ценности события

КОНТЕКСТ

Kafka — не источник истины: сообщение уже в Cassandra, теряются лишь производные события.

РЕШЕНИЕ

Доставка — 48 ч для индексатора; статусы, межрегион и push — 6 ч; acl.invalidate — 7 сут.

КОМПРОМИСС

Сбой ES дольше 48 ч — доиндексация из Cassandra.

Пороги устаревания событий

КОНТЕКСТ

После сбоя потребителя накопленный поток хлынул бы разом: миллионы доставок и push.

РЕШЕНИЕ

Доставка старше 10 мин — один sync_hint на чат; push старше 30 мин отбрасывается.

КОМПРОМИСС

При отставании > 10 мин сообщение видно после синхронизации, а не мгновенно.

Хранилища и выбор БД

Хранилище	Что хранит	Почему выбрано	Репликация	Хранение
Cassandra	Сообщения, курсоры, контакты, указатели на архив	Запись 2,65 млн/сек, рост добавлением нод	RF3 в каждом регионе, ×9 копий	12–13 мес., ~58 ПБ; старше — в архив
Архив истории	Сообщения старше года, сегменты ~256 МБ	Читается редко, сжатие zstd ×0,4	Erasure coding ×1,4 + бэкап в Deep Archive	Бессрочно, +1,3 ПБ в год
Object Storage	Фото, видео, голос, файлы до 2 ГБ	7,5 ЭБ к 3-му году — S3 не сходится	Erasure coding ×1,4 + суточный бэкап	Бессрочно; CDN-кэш 14 дней
Aurora (PostgreSQL)	Профили, сессии, группы и членство	Уникальность телефона и тега, ACID	Один writer + реплики в 2 регионах	Бессрочно, ~8 ТБ
DynamoDB Global Tables	Регионы устройств, «в сети», push-токены	Глобальная запись и чтение, managed	Multi-master во всех регионах	Бессрочно, ~3 ТБ
Elasticsearch	Полнотекстовый индекс сообщений	Поиск, которого нет в Cassandra	Кластер на регион, без копии между ними	1 год: hot 14 дн., warm, cold; ~4,2 ПБ
Kafka	События доставки, статусов, push	Буфер и повтор доставки	RF3 в регионе, между регионами нет	Доставка 48 ч, прочее 6 ч, acl 7 сут.
Redis	Соединения, push-состояние, счётчики, кэши, подписки на статус	Миллионы операций/сек в памяти	Мастер + реплика, только в регионе	Эфемерно: TTL, восстанавливается

Кэширование: где и почему

Что кэшируем	Где	Зачем	Свежесть
Медиафайлы	CDN + edge-кэш	15,3 Тбит/с отдачи; 94% — группы: одно фото качают ~30 человек	Тёплое окно 14 дней; подпись ссылки 24 ч
Хвост каналов и больших групп	Redis tail:{chat_id}	Канал на 100 млн читают из одной партиции сразу после поста	TTL 5 сек, один запрос в Cassandra на промах
Членства пользователя	Redis acl:{user_id}	374 тыс. проверок доступа/сек — Aurora видит только промахи	Сброс при изменении; TTL 1 ч, не больше 24 ч
Участники малой группы	Redis cache:group:{id}	Список нужен на каждое сообщение группы до 1 000 участников	Сброс при любом изменении состава
Страницы архива	Память Archive Service	Повторное чтение старой истории не идёт в Object Storage	Данные неизменяемы — только вытеснение (LRU)

Не кэшируем: сообщения личных чатов и малых групп — у чата мало читателей, а новое сообщение должно быть видно сразу; профили — 2,6 тыс. чтений/сек, Aurora справляется сама.

ИТОГИ РАСЧЁТОВ

Ресурсы и стоимость

Ноды и инстансы, с запасом на отказ региона

Компонент	Штук
Cassandra (Message Store)	14 520
Messaging Service	651
Elasticsearch	393
Connection Gateway	297
Redis	210
Прочие сервисы	~123
Group & Channel	72
Kafka	42
Aurora (PostgreSQL)	15–21

Стоимость в месяц

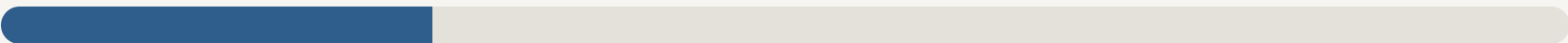
\$137,2 млн

≈ \$0,27 на активного пользователя в месяц

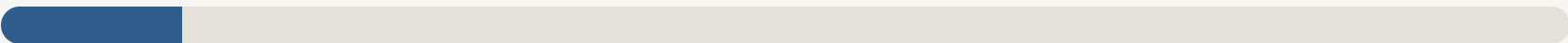
Network — отдача медиа · \$82,8 млн · 60,3%



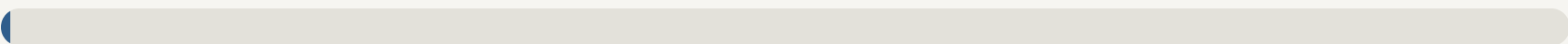
Storage* — хранение медиа · \$37,7 млн · 27,5%



Compute — серверы · \$15,9 млн · 11,6%



Managed — Redis, Aurora, DynamoDB · \$0,8 млн · 0,6%



88% счёта — медиа, а не сообщения

* Storage посчитан по тарифам S3 для наглядности; своя система хранения дешевле, но вне scope

ВЫВОДЫ

Что определяет систему

88%

счёта — медиа: отдача и хранение (по тарифам S3; своя система — дешевле, вне scope)

×19

меньше push после агрегации — FCM загружен на 84%

0,1%

счёта стоит запас ×1,5 — отказ региона не становится глобальным

\$0,27

в месяц на активного пользователя, всего \$137,2 млн/мес

Дальнейшая проработка: ключи KMS на горячем пути, разложение SLA 99,95% по компонентам, процесс удаления неактивных аккаунтов